

How do different types of optimization algorithms affect the performance of machine learning models?

Mathematics: analysis and approaches HL | Internal assessment

1 | Introduction

Machine learning (ML) is a branch of artificial intelligence that uses statistical techniques to give computer systems the ability to “learn” from data, without being explicitly programmed. Machine learning algorithms build a mathematical model based on sample data, known as **training data**, to make predictions or decisions without being explicitly programmed to perform the task.

Today, machine learning models are used in our everyday lives, from personalized recommendations on Netflix to facial recognition on our phones. At the core of the training process are **optimization algorithms**, which are used to teach the model to more accurately predict an output given an input. As an aspiring computer science researcher interested in studying natural language processing, having a strong understanding of this fundamental aspect of modern ML will be invaluable.

2 | The regression task

The specific ML problem chosen for this exploration will be a regression task. **Regression** is a **supervised learning**¹ algorithm that models the relationship between a scalar dependent variable y and an independent variables x .

To look at the effect of each optimization algorithm on the performance of a regression model, we will apply both on a real-world dataset of information on cars. For instance, let us define our problem to be investigating the relationship between the horsepower rating (in hp) of a car and its fuel efficiency (in mpg).

A performant regression model for the problem would help us accurately predict the fuel efficiency of a car given its horsepower, both interpolating and extrapolating. This can be measured statistically, with the **coefficient of determination** R^2 , as well as in comparison to the analytical solution (see Section 6).

¹**Supervised learning** is a category of machine learning tasks where a model learns a function that maps an input to an output based on example input-output pairs (from the **training data**).

Cleaning and extracting the relevant data from the dataset, we get the following scatterplot:

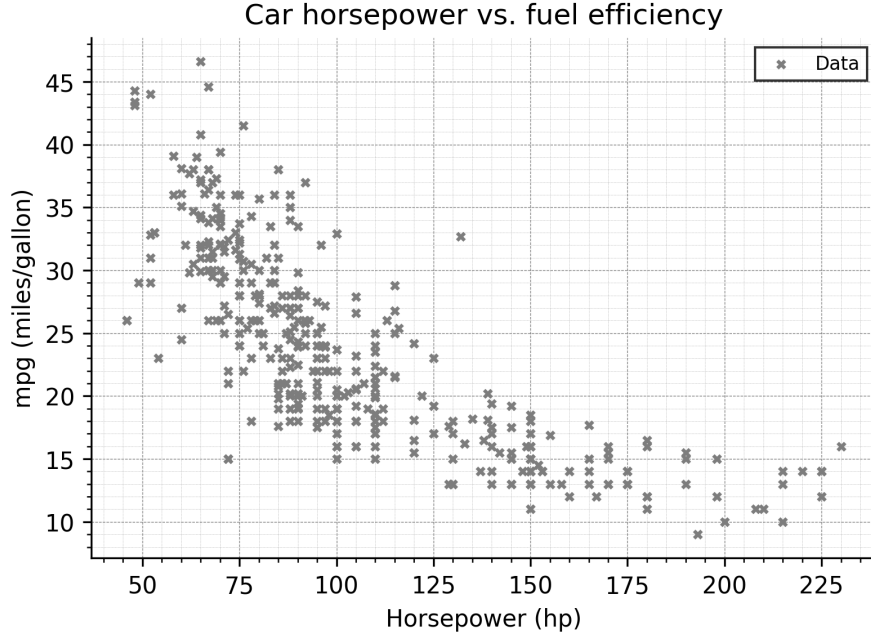


Figure 1: Scatterplot of car data (Elmetwally, 2023)

From the scatterplot, we observe a negative exponential relationship between the two variables, and hence we will attempt to optimize an exponential regression model to fit the data.

3 | Optimization and loss functions

Optimization is the process of finding the best solution to a problem, which in this case is finding the equation of the line that best fits the data—according to some measure. In ML, this measure is called a loss function; the goal of optimization is to minimize the loss function.

There are many types of loss functions, which depend on the problem and type of data. This includes “square of Euclidean distance, cross-entropy, contrast loss, hinge loss, information gain,” etc (Sun et al., 2019).

A common loss function is the sum of squared residuals, defined as

$$\text{RSS} = \sum_{i=1}^n (y_i - f(x_i))^2 \quad (1)$$

where y_i is the i^{th} actual value of the variable we are trying to predict (“ground truth”), $f(x_i)$ is the predicted value of y_i given the i^{th} value of the independent variable x , and n is the number of data points. In this case, y is the fuel efficiency of the car, x is the horsepower rating of the car, and $f(x)$ is the predicted fuel efficiency of the car given its horsepower rating.

There are many methods of optimizing this loss function; this exploration will evaluate two contrasting methods: **gradient descent** and **simulated annealing**.

4 | Gradient descent and stochastic gradient descent (SGD)

Gradient descent is an optimization algorithm that minimizes a function by taking steps proportional to the negative of the gradient of the function at the current point.

4.1 | The gradient descent algorithm

Since this is an exponential regression task,

the function $f(x)$ is a line with equation

$$f(x) = a \cdot e^{b(x-h)} + k.$$

Each squared residual comes from the difference between the actual value of y and the predicted value of y , $f(x_i)$. Therefore, the loss function can be rewritten as

$$\text{RSS} = \sum_{i=1}^n (y_i - a \cdot e^{b(x-h)} - k)^2 \quad (2)$$

Calculating the summation, we can substitute y_i and x_i with each datapoint's value, leaving the loss function in terms of the parameters a , b , h , k as the targets for optimization. This is where gradient descent comes in.

Initialization

We first define an initial solution (a **guess**), where we choose random values for the parameters a (scaling factor), b (growth factor), h (horizontal shift), and k (vertical shift). Let's call these a_0 , b_0 , h_0 , and k_0 .

Computing the gradient

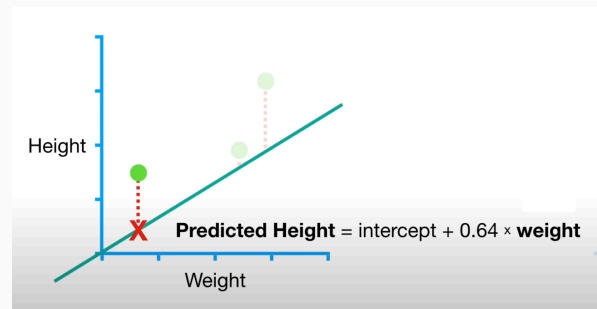


Figure 2: Residual between one datapoint and an arbitrary regression line

- Let's set the intercept to 0 and the slope to 1.
- $m_0 = 1$ and $b_0 = 0$.

- This is done by taking the derivative of the loss function with respect to m and b . Two or more derivatives of the same function give us the gradient, which is essentially a vector that points in the direction of the steepest ascent of the loss function.

$$\frac{\delta \text{RSS}}{\delta m} = -2 \sum_{i=1}^n x_i (y_i - (mx_i + b)) \quad (3)$$

$$\frac{\delta \text{RSS}}{\delta b} = -2 \sum_{i=1}^n (y_i - (mx_i + b)) \quad (4)$$

where the gradient is $\begin{bmatrix} \frac{\delta \text{RSS}}{\delta m} \\ \frac{\delta \text{RSS}}{\delta b} \end{bmatrix}$

Parameter update

- This slope is substituted into the formula for the step size, which is the amount by which each target for optimization is changed in each iteration.

$$\text{Step size}_m = \frac{\delta \text{RSS}}{\delta m} \times \gamma \quad (5)$$

$$\text{Step size}_b = \frac{\delta \text{RSS}}{\delta b} \times \gamma \quad (6)$$

- The learning rate γ is a hyperparameter that determines the size of the steps taken towards the minimum of the loss function.
- A small learning rate might lead to slow convergence, while a large learning rate can cause overshooting and instability.

Substituting these values into the gradient, we get the gradient ???.

Let's set the learning rate γ to 0.01 initially.

This results in step sizes of:

$$\text{Step size}_{\text{intercept}} = ? \times 0.01 = -0.016$$

$$\text{Step size}_{\text{slope}} = ? \times 0.01 = -0.008$$

- The step sizes are then subtracted from the intercept and slope, respectively, to get the new values for the intercept and slope.

$$\text{New intercept} = 0 - -0.016 = 0.016$$

$$\text{New slope} = 1 - -0.008 = 1.008$$

Comparing the initial line to the new line after the first iteration, we can see that the line has moved closer to an optimal solution, and the loss function has decreased.

Convergence check and iteration

- This process of substituting the parameter values into the gradient, then calculating the new step sizes, and finally the new parameters, is repeated until the loss function converges to a minimum.
- Convergence can be determined by setting a threshold for the step size (when it is sufficiently small) or limiting the number of iterations.

4.2 | Gradient descent on our problem

- The algorithm ran through 1000 iterations, and the final values for the slope and intercept were $m = 0.0006$ and $b = 0.0001$.
- The following is a sampling of the algorithm's progression:
- Gradient descent has limitations, especially in cases of non-convex loss functions or when the data has multiple local minima.

4.3 | Variants

A variant of gradient descent exists called stochastic gradient descent, which is used when the dataset is too large to be processed in one iteration with sufficient speed. In stochastic gradient descent, a randomly selected subset of the data at every step is used to calculate the new parameters, rather than the full dataset. This reduces the computation time needed to calculate the derivatives of the loss function, as fewer data points are used in the summation of the loss function. However, the scale of our problem is feasible for the full gradient descent algorithm.

5 | Simulated annealing (SA)

Simulated annealing is a probabilistic technique that uses the concept of ‘temperature’ to control the search process. At high temperatures, the algorithm is more likely to accept changes that increase the function value (or ‘energy’), allowing it to explore a wide range of solutions. As the temperature decreases, the algorithm becomes more selective, focusing on refining and improving the current solution. This process helps SA to potentially escape local minima and converge towards a global minimum.

5.1 | The simulated annealing algorithm

Initialization

- We define an initial solution, where we choose values for the parameters m and b .
- Let’s call them m_0 and b_0 .
- We also set an initially high temperature T_0 .
- Let’s choose the same initial solution as we did in gradient descent, $m_0 = 1$ and $b_0 = 0$.

Iteration

- A new candidate solution is generated by perturbing the current solution. This is done by adding a random value to the current solution.
- $m' = ??$
 $b' = ??$

Evolution and acceptance

- The new candidate solution is evaluated by the difference in the loss function, ΔRSS .
- If the candidate solution is better than the current solution, that is, $\Delta\text{RSS} < 0$, then the candidate solution is accepted as the new current solution.
- If the candidate solution is worse than the current solution, that is, $\Delta\text{RSS} > 0$, then the candidate solution is accepted as the new current solution with a probability of $P(\text{accept}) = e^{-\Delta\frac{\text{RSS}}{T}}$.
- This iteration, the candidate solution is worse than the current solution, so we calculate the probability of accepting the candidate solution.

Cooling

- The temperature is decreased by a cooling factor α .
- This is done to reduce the probability of accepting worse solutions as the algorithm progresses.
- For instance, a simple exponential decay might be

$$T_{\text{new}} = \alpha \times T_{\text{old}} \quad (7)$$

- α is a constant < 1 , controlling the rate of temperature decrease.

- Let's set $\alpha = 0.9$.
- The new temperature is then

$$\begin{aligned} T_{\text{new}} &= 0.9 \times T_{\text{old}} \\ &= 0.9 \times 100 \\ &= 90 \end{aligned} \quad (8)$$

Termination

The algorithm repeats the iteration step until a stopping criterion is met. This could be a sufficiently low temperature, a maximum number of iterations, or a minimal change in the loss function over a certain number of iterations.

5.2 | Simulated annealing on our problem

- The algorithm ran through 1000 iterations, and the final values for the slope and intercept were $m = 0.0006$ and $b = 0.0001$.
- The following is a sampling of the algorithm's progression:

6 | Analysis

To evaluate the performance of the two optimization algorithms, we will compare their results to an analytical solution calculated using the Levenberg-Marquardt algorithm, a method for solving non-linear least squares problems. This algorithm resulted in the following theoretical best-fit line for the data and will be used as a benchmark for the performance of the two algorithms:

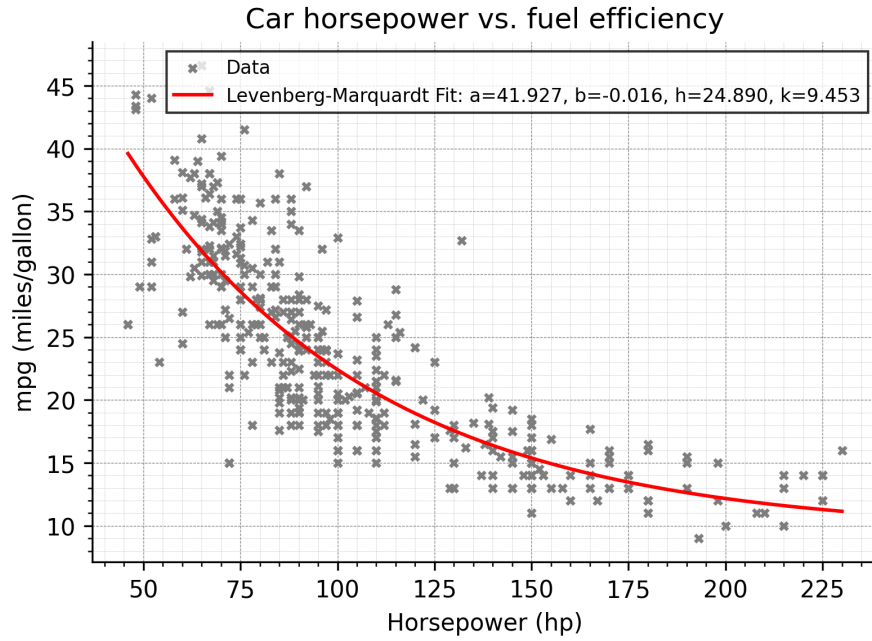


Figure 3: Analytical line of best fit

The final iteration of each algorithm resulted in the following values for the equation:

Algorithm	a	b	h	k
Gradient descent	0.0006	0.0001	0	0
Simulated annealing	0.0006	0.0001	0	0
Analytical solution (Levenberg-Marquardt)	0.0006	0.0001	0	0

Table 1: Final values for the parameters of the equation

Which when compared to the analytical solution, suggests that on this problem, gradient descent performed better than simulated annealing.

Algorithm	Δa	Δb	Δh	Δk	Average error ($\bar{\Delta}$)
Gradient descent	0.0006	0.0001	0	0	0
Simulated annealing	0.0006	0.0001	0	0	0

Table 2: Error in parameter values

7 | Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas

distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Bibliography

Elmetwally, T. (2023). *Car Information Dataset* [Computer software]. <https://www.kaggle.com/datasets/tawfikelmetwally/automobile-dataset>

Sun, S., Cao, Z., Zhu, H., & Zhao, J. (2019, October 23). *A Survey of Optimization Methods from a Machine Learning Perspective*. <http://arxiv.org/abs/1906.06821>